Vijay Academy Presents

Savitribai Phule Pune University (SPPU)

Artificial Intelligence (AI)

TE Computer Engineering (2019 Pattern)

One Shot Lecture

Unit 3. Adversarial Search and Games

What You Get : Strategy For Exam :

- Simple Explanation
- PPT + Notes
- PYQs + Most IMP Questions

Subscribe The Vijay Academy For Free Engineering Content

Unit 3. Adversarial Search and Games

Syllabus Topics :

1. Game Theory
2. Optimal Decisions in Games (Minimax Algorithm)
3. Heuristic Alpha-Beta Tree Search
4. Monte Carlo Tree Search (MCTS)
5. Stochastic Games
6. Partially Observable Games
7. Limitations of Game Search Algorithms
8. Constraint Satisfaction Problems (CSP)
9. Constraint Propagation: Inference in CSPs
10. Backtracking Search for CSPs

1. Game Theory

Definition :

Game Theory is the study of **how two or more players make decisions** when the outcome of each player depends on what the **other players do.**

In AI, we use it to make computers **play games smartly** against opponents.

Key Points :

There are **2 players** — MAX and MIN

MAX wants to **win** (maximize score)

MIN wants to **block** (minimize MAX's score)

Each player plays **optimally** (best possible move)

The game is represented as a **tree** called **Game Tree**

Each node = a **game state**

Each branch = a **possible move**

Leaf nodes = **final scores** (called utility values)

1. Game Theory

Example

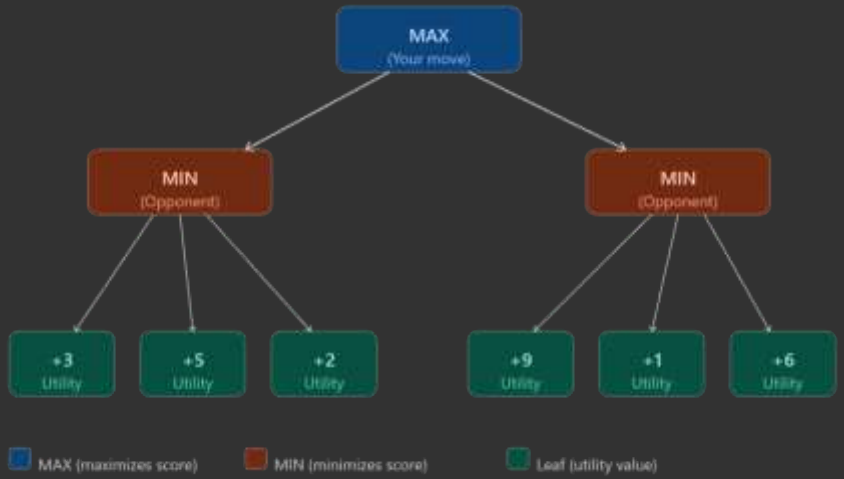
Think of **Tic-Tac-Toe:**

You are MAX (X)

Opponent is MIN (O)

Both play their best moves

AI explores all possible moves and picks the best one.



2. Optimal Decisions in Games (Minimax Algorithm)

Definition

Minimax is an algorithm used in 2-player games where:

MAX tries to **maximize** the score

MIN tries to **minimize** the score

Both play **perfectly/optimally**

The algorithm **goes deep into the tree**, finds the best move by going bottom to top.

How it works (Step by Step) :

Build the full game tree

Assign utility values to all leaf nodes

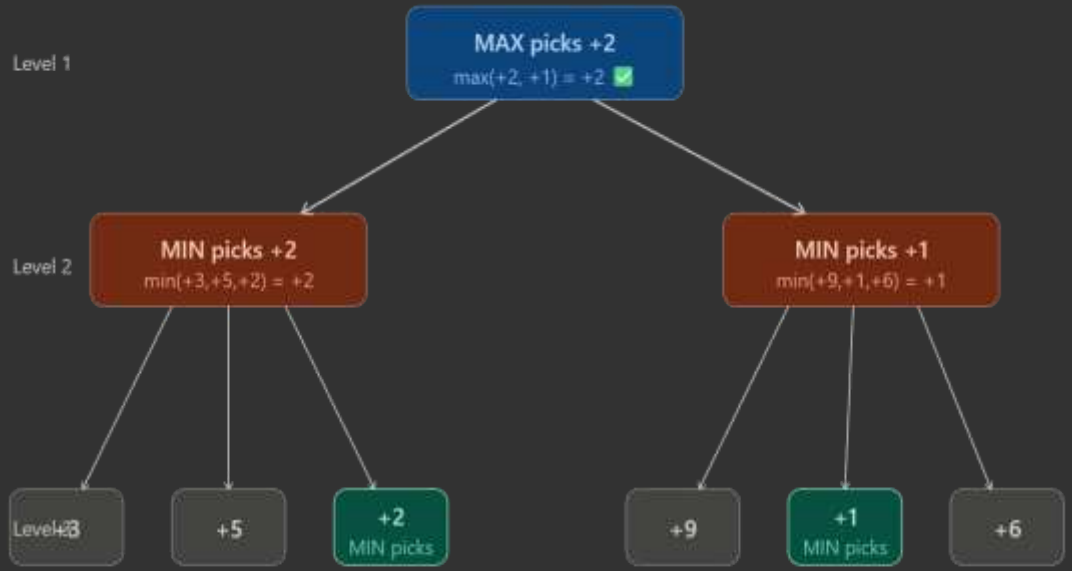
Go **bottom to top**

At MIN level → pick **minimum** value

At MAX level → pick **maximum** value

The value that reaches the root = best move

2. Optimal Decisions in Games (Minimax Algorithm)



ℹ Values propagate from bottom (leaves) to top (root)

5. Stochastic Games

Definition :

Stochastic means **random.**

A Stochastic Game is a game where **luck/chance is involved** along with player decisions.

Example → **Rolling a dice in Ludo or Snake & Ladders**

Normal Game vs Stochastic Game

In **normal game (like Chess):**

You make a move → exactly that happens

No randomness

In **stochastic game (like Ludo):**

You roll dice first → random number comes

Then you make a move based on that number

Outcome is **not fully in your control**

5. Stochastic Games

How AI handles it?

In Minimax we had only MAX and MIN nodes.

In Stochastic Games we add one more node called **CHANCE node.**

Expectiminimax Algorithm

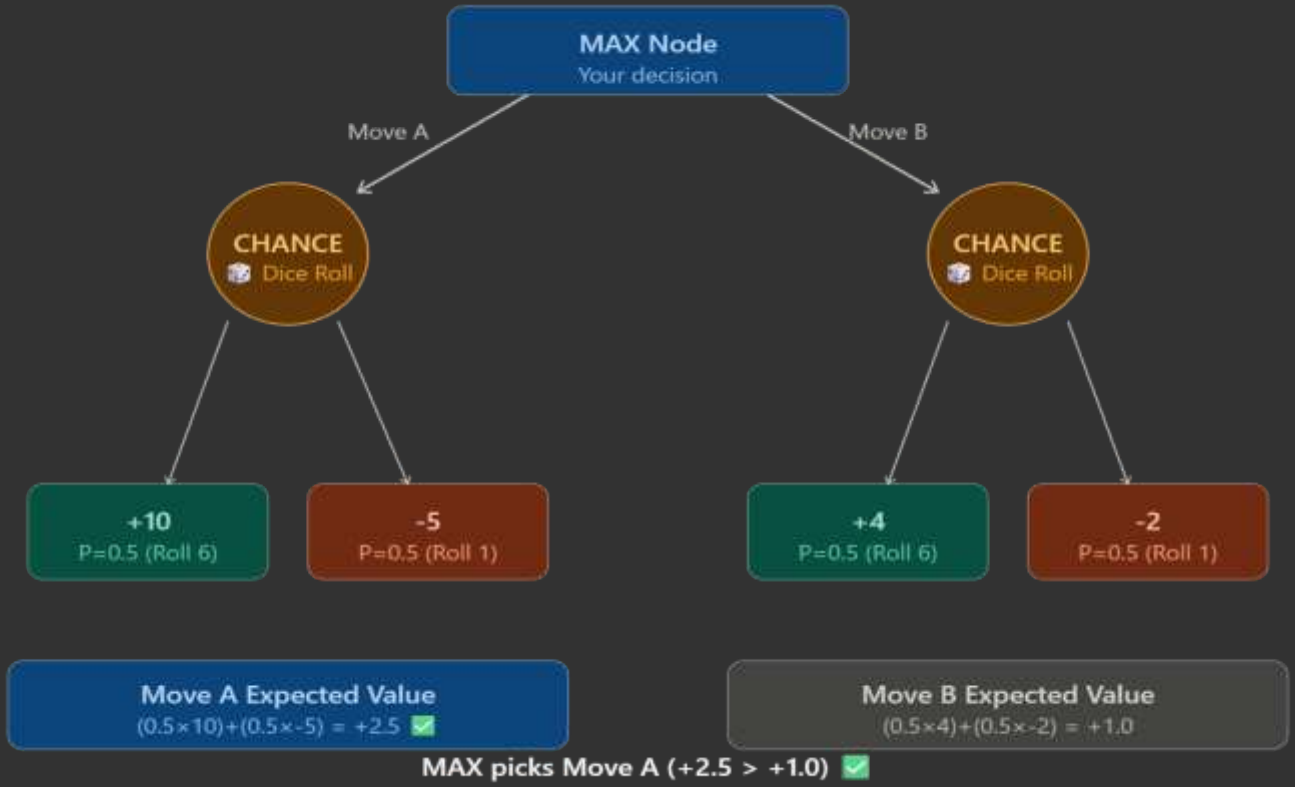
Same as Minimax BUT adds chance nodes

At MAX node → pick **maximum**

At MIN node → pick **minimum**

At CHANCE node → calculate **expected value**

Expected Value = Σ (probability × utility value)


MAX Node
Your decision
Move A
Move B
CHANCE
Dice Roll
CHANCE
Dice Roll
+10
P=0.5 (Roll 6)
-5
P=0.5 (Roll 1)
+4
P=0.5 (Roll 6)
-2
P=0.5 (Roll 1)
Move A Expected Value
(0.5×10)+(0.5×-5) = +2.5
Move B Expected Value
(0.5×4)+(0.5×-2) = +1.0
MAX picks Move A (+2.5 > +1.0)

Blue Box at Top — MAX Node
You have 2 choices → **Move A** or **Move B**
You don't know what dice will give yet
So you pick based on **expected value.**

Yellow Circles — CHANCE Nodes
After you pick a move, **dice is rolled**
Left circle = chance after **Move A**
Right circle = chance after **Move B**
These are random → you cannot control them

Green Boxes — Good Outcomes (+10 and +4)
If dice gives **6** → you get good result
Move A gives **+10**
Move B gives **+4**
Probability = **0.5** (50% chance)

Red Boxes — Bad Outcomes (-5 and -2)
If dice gives **1** → you get bad result
Move A gives **-5**
Move B gives **-2**
Probability = **0.5** (50% chance)

Bottom Left Box — Move A Calculation
Expected Value = (0.5 × 10) + (0.5 × -5)
= 5 + (-2.5)
= **+2.5**

Bottom Right Box — Move B Calculation
Expected Value = (0.5 × 4) + (0.5 × -2)
= 2 + (-1)
= **+1.0**

Final Decision
Move A gives **+2.5** Move B gives **+1.0**
MAX picks **Move A** because 2.5 > 1.0

6. Partially Observable Games

Definition :

A Partially Observable Game is a game where **you cannot see the full game state.**

You only see **part of the information** — rest is hidden!

Real life example → **Card games like Poker**

You can see YOUR cards

But you CANNOT see opponent's cards

You make decisions based on **incomplete information.**

Fully Observable vs Partially Observable

Fully Observable:

You can see EVERYTHING

Example → Chess, Tic-Tac-Toe

Easy for AI to decide

Partially Observable:

You can see only SOME things

Example → Poker, Battleship game

Hard for AI → must **guess** hidden info

6. Partially Observable Games

How AI handles it?

AI maintains a **Belief State**

Belief State = AI's best guess about what the hidden information might be

Example in Poker:

AI sees its own cards

AI **guesses** opponent's cards based on their behavior

AI picks best move based on that guess.

6. Partially Observable Games

Point	Stochastic Games	Partially Observable Games
Meaning	Randomness due to chance events	Randomness due to hidden information
Problem	Dice/cards add uncertainty	You can't see full game state
Example	Ludo, Backgammon	Poker, Battleship
AI handles by	Expectiminimax algorithm	Belief State
Extra node in tree	CHANCE node added	No chance node — uses probability
Control	Partly in your control	Fully in your control but with incomplete info
Randomness source	External event (dice roll)	Opponent's hidden info

7. Limitations of Game Search Algorithms

Limitations :

1. Huge Search Space

Real games like Chess have **billions of possible moves.**
Impossible to explore every single branch.

2. Time Problem

Exploring deep trees takes **too much time.**
In real games you have a **time limit** to make a move.

3. Heuristic is not always accurate

When we cannot go to leaf nodes
---we use heuristic function.
Heuristic is just an **estimate** — not always correct.
Wrong heuristic = wrong decision.

4. Cannot handle Randomness well

Real games have dice, cards, random events.
Standard Minimax fails in stochastic games.

5. Hidden Information problem

Minimax needs to **see full game state.**
In Poker or Battleship — information is hidden.
Algorithm struggles with partially observable games.

6. Multi-player problem

Minimax works for only **2 players.**
Real world games can have **3 or more players.**
Algorithm becomes very complex with more players.

7. Memory problem

Storing the entire game tree needs **huge memory.**
Not practical for complex games.

8. Constraint Satisfaction Problems (CSP)

Definition :

CSP is a problem where you have to **assign values to variables** such that all **constraints (rules) are satisfied.**

Think of it like filling a form where every field must follow certain rules!

3 Main Components of CSP :

1. Variables (X)

Things we need to assign values to

Example → X1, X2, X3

2. Domain (D)

Set of possible values each variable can take

Example → {Red, Green, Blue}

3. Constraints (C)

Rules that must be satisfied

Example → X1 ≠ X2 (neighboring regions cannot have same color)

8. Constraint Satisfaction Problems (CSP)

Example — Map Coloring Problem

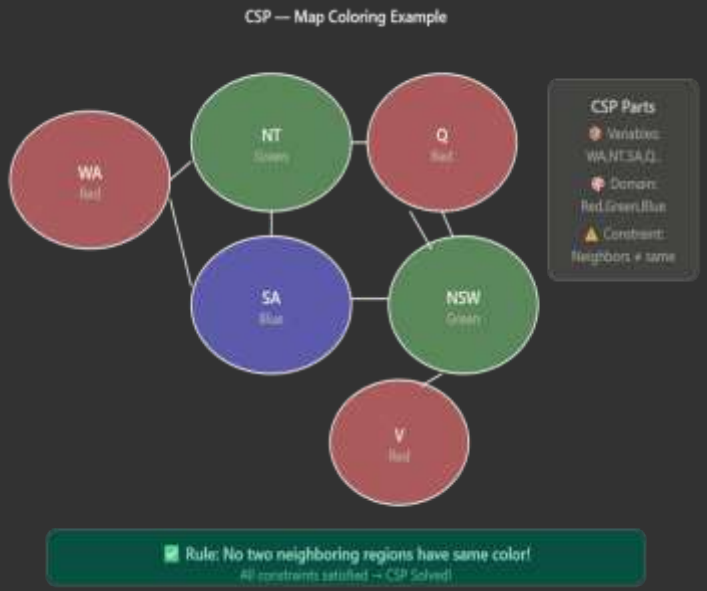
Problem:

Color a map with 3 colors
No two **neighboring** regions can have same color

Variables → WA, NT, SA, Q, NSW, V (regions of Australia)

Domain → {Red, Green, Blue}

Constraint → Neighboring regions must have different colors



8. Constraint Satisfaction Problems (CSP)

Types of Constraints

1. Node Consistency

What it means:

Check ONE variable at a time.

Remove values from its domain that break its OWN rules.

Simple Example :

Variable → SA (South Australia)

Domain → {Red, Green, Blue}

Rule → SA ≠ Red

So remove Red from domain

New Domain → {Green, Blue}

8. Constraint Satisfaction Problems (CSP)

2. Arc Consistency

What it means:

Check TWO variables at a time.

For every value of X1 → there must be at least one valid value in X2.

If no valid value found in X2 → remove that value from X1.

Simple Example:

X1 = WA → Domain {Red, Green, Blue}

X2 = NT → Domain {Red, Green, Blue}

Rule → WA ≠ NT

WA = Red → NT can be Green or Blue

WA = Green → NT can be Red or Blue

WA = Blue → NT can be Red or Green

8. Constraint Satisfaction Problems (CSP)

3. Path Consistency

What it means:

Check THREE variables at a time.

X1 and X3 are consistent → but is there a X2 in between that satisfies both?

Simple Example:

X1 = Red, X3 = Red

Rule → all neighbors must be different.

We need X2 between them

X2 cannot be Red (neighbors of both X1 and X3)

X2 = Green or Blue

Path is consistent!

9. Constraint Propagation: Inference in CSPs

Definition :

Constraint Propagation means Simply → when you fix one variable's value,
it **automatically eliminates** invalid values from other variables!

Think of it like **Sudoku** — when you write 5 in one box, you automatically know 5 cannot appear in that row, column, or box!

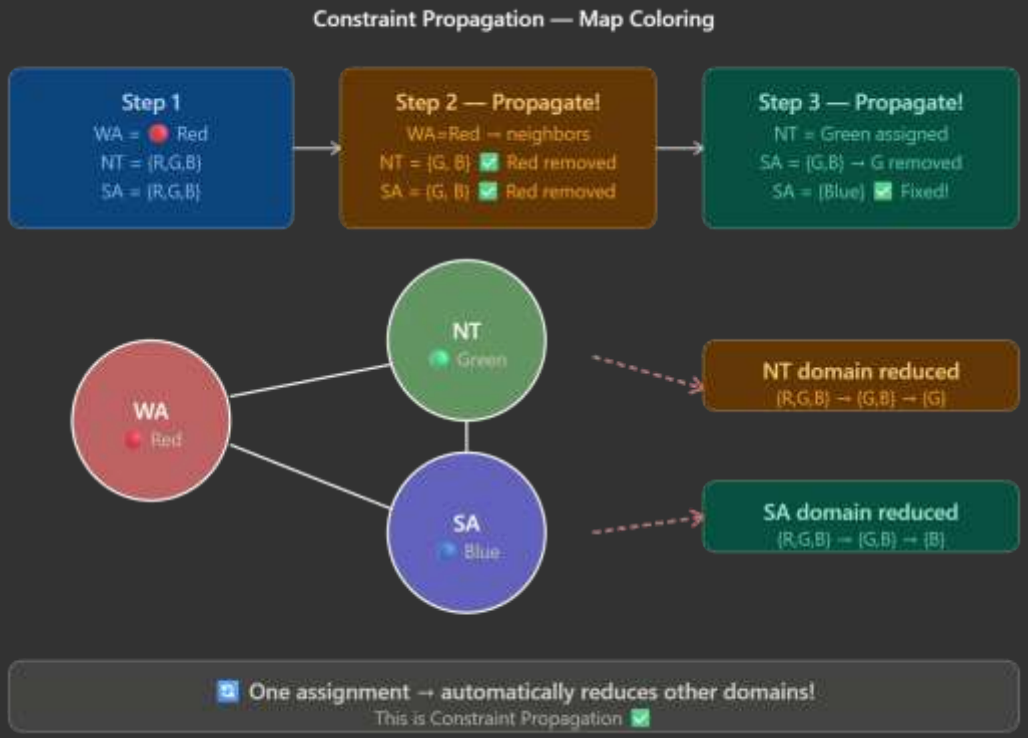
Why do we need it?

Without Constraint Propagation:
AI tries ALL possible combinations → very slow

With Constraint Propagation:
AI reduces choices early → much faster

9. Constraint Propagation: Inference in CSPs

Example — Map Coloring :



Thank You!

Like, Share & Subscribe

https://youtube.com/@thevijayacademy?si=uYwlDuAxyZfLsiei

Also join Telegram channel for updates and notes :

https://t.me/the_vijay_academy